Memoria Práctica 2: MPI - Conjunto de Mandelbrot

Arquitectura y Computación de Altas Prestaciones

Ignacio Palomares Martín

1. Introducción

En esta segunda práctica, el objetivo es dar un paso más con MPI y paralelizar la generación de una imagen de un fractal clásico: el conjunto de Mandelbrot.

La base matemática es bastante intuitiva: cogemos cada píxel de la imagen, lo pasamos a unas coordenadas ***(cx, cy)*** y le aplicamos una fórmula en bucle ***(Zn+1 = Zn^2 + c)*** partiendo de cero. Si el valor no se dispara y no pasa de un radio de 2 después de un número máximo de iteraciones (max_iter), decimos que pertenece al fractal y lo pintamos de negro. Si se escapa antes, le damos un color RGB dependiendo de cuántas iteraciones haya tardado en salir. Como el cálculo de un píxel no depende para nada del resultado de sus vecinos, es un problema ideal para ser paralelizado.

2. Implementación y Paralelización

Algoritmo Secuencial:

En la primera versión (la secuencial), un único proceso se encarga de ir recorriendo todos los píxeles de la imagen. He usado una resolución 4K para que los cálculos pesen un poco más y las diferencias de tiempo se noten de verdad. Un bucle for principal convierte la posición lineal en coordenadas (x, y) y luego el bucle while se encarga de la carga pesada de iterar la fórmula de Mandelbrot. Los colores se van guardando en un array dinámico en memoria y, al terminar de medir el tiempo, usamos la librería stb_image_write (que descargamos asi: wget https://raw.githubusercontent.com/nothings/stb/master/stb_image_write.h) para exportarlo a un JPG.

- **Compilación:** g++ mandelbrot_secuencial.cpp -o secuencial -O3
- **Ejecución:** ./secuencial

```cpp
#include <iostream>
#include <chrono>

#define STB_IMAGE_WRITE_IMPLEMENTATION
#include "stb_image_write.h"

int main() {
    // usamos una resolucion alta (4k) para que tarde un poco y se vea mejor el resultado
    int width = 3840;
    int height = 2160;
    int channels = 3; // rgb
    int max_iter = 1000; // maximo de iteraciones por pixel

    int total_pixeles = width * height;
    unsigned char* img_data = new unsigned char[total_pixeles * channels];

    std::cout << "[INFO] Generando fractal en 4K (Secuencial)... paciencia." << std::endl;

    // aqui empieza el cronometro
    auto start_time = std::chrono::high_resolution_clock::now();

    for (int i = 0; i < total_pixeles; i++) {
        // pasar la posicion lineal i a x e y
        int x = i % width;
        int y = i / width;

        // llevar las coordenadas al plano de mandelbrot
        float cx = -2.5f + 3.5f * (float)x / width;
        float cy = -1.0f + 2.0f * (float)y / height;

        float zx = 0.0f, zy = 0.0f;
        int iter = 0;

        // el bucle pesado de mandelbrot
        while (zx * zx + zy * zy < 4.0f && iter < max_iter) {
            float temp = zx * zx - zy * zy + cx;
            zy = 2.0f * zx * zy + cy;
            zx = temp;
            iter++;
        }

        // poner el color segun cuanto tardo en escapar
        int idx = i * 3;
        if (iter == max_iter) {
            // no escapo: queda negro
            img_data[idx] = 0; img_data[idx+1] = 0; img_data[idx+2] = 0;
        } else {
            // escapo: le damos color segun las iteraciones
            img_data[idx]   = (iter * 5) % 256;  // rojo
            img_data[idx+1] = (iter * 7) % 256;  // verde
            img_data[idx+2] = (iter * 11) % 256; // azul
        }
    }

    // aqui acaba el cronometro
    auto end_time = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> diff = end_time - start_time;

    // guardar imagen y limpiar
    stbi_write_jpg("secuencial_mandelbrot.jpg", width, height, channels, img_data, 100);
    delete[] img_data;

    std::cout << "Tiempo secuencial: " << diff.count() << " segundos" << std::endl;
    return 0;
}
```

Estrategia de Paralelización (MPI):

A la hora de paralelizar esto con MPI, el enfoque cambia un poco respecto a la práctica de calcular el número Pi. Como aquí estamos montando una imagen que necesita estar perfectamente ordenada en memoria, no nos sirve ir intercalando iteraciones. Lo que he hecho es dividir el número total de píxeles entre los procesos que tengamos ***(pixeles_locales = pixeles_totales / size)*** y darle a cada uno un bloque continuo de la imagen.

El otro cambio importante es cómo juntamos los resultados. La función MPI_Reduce ya no nos vale porque no queremos sumar números, sino concatenar bloques de memoria. Por eso, una vez que cada proceso llena su buffer_local con los píxeles que le han tocado, utilizamos la función colectiva **MPI_Gather**. Lo que hace esta función es coger los fragmentos de todos los procesos y ensamblarlos en orden dentro del array final ***(img_resultado)*** del proceso 0, que es quien guarda la foto.

- **Compilación:** mpicxx -Wall -O3 mandelbrot_paralelo.cpp -o paralelo
- **Ejecución:** mpirun -np <numero_procesos> ./paralelo

```cpp
#include <iostream>
#include <vector>
#include <mpi.h>

#define STB_IMAGE_WRITE_IMPLEMENTATION
#include "stb_image_write.h"

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int width = 3840, height = 2160;
    int channels = 3;
    int max_iter = 1000;

    int pixeles_totales = width * height;

    // si no se puede partir bien, mejor parar aqui
    if (pixeles_totales % size != 0) {
        if (rank == 0) std::cerr << "Error: Píxeles no divisibles entre procesos." << std::endl;
        MPI_Finalize();
        return 1;
    }

    int pixeles_locales = pixeles_totales / size;
    int datos_locales = pixeles_locales * channels; // 3 bytes por píxel (RGB)

    std::vector<unsigned char> buffer_local(datos_locales);
    unsigned char* img_resultado = nullptr;

    if (rank == 0) {
        img_resultado = new unsigned char[pixeles_totales * channels];
        std::cout << "[INFO] Generando fractal en 4K (Paralelo) con " << size << " procesos..." << std::endl;
    }

    // aqui empieza el tiempo del paralelo
    double inicio = MPI_Wtime();

    // cada proceso calcula su trozo de la imagen
    for (int i = 0; i < pixeles_locales; i++) {
        // ver que pixel global me toca segun mi rank
        int pixel_global = (rank * pixeles_locales) + i;

        int x = pixel_global % width;
        int y = pixel_global / width;

        float cx = -2.5f + 3.5f * (float)x / width;
        float cy = -1.0f + 2.0f * (float)y / height;

        float zx = 0.0f, zy = 0.0f;
        int iter = 0;

        while (zx * zx + zy * zy < 4.0f && iter < max_iter) {
            float temp = zx * zx - zy * zy + cx;
            zy = 2.0f * zx * zy + cy;
            zx = temp;
            iter++;
        }
```

```
        int idx = i * 3; // indice dentro del buffer local
        if (iter == max_iter) {
            buffer_local[idx] = 0; buffer_local[idx+1] = 0; buffer_local[idx+2] = 0;
        } else {
            buffer_local[idx]   = (iter * 5) % 256;
            buffer_local[idx+1] = (iter * 7) % 256;
            buffer_local[idx+2] = (iter * 11) % 256;
        }
    }

    // el proceso 0 recoge todo lo que calcularon los demas
    MPI_Gather(buffer_local.data(), datos_locales, MPI_UNSIGNED_CHAR,
               img_resultado, datos_locales, MPI_UNSIGNED_CHAR,
               0, MPI_COMM_WORLD);

    // aqui acaba el tiempo del paralelo
    double fin = MPI_Wtime();

    if (rank == 0) {
        stbi_write_jpg("paralelo_mandelbrot.jpg", width, height, channels, img_resultado, 100);
        delete[] img_resultado;
        std::cout << "Tiempo paralelo: " << (fin - inicio) << " segundos" << std::endl;
    }

    MPI_Finalize();
    return 0;
}
```

3. Entorno de Ejecución y Metodología

Para comprobar el rendimiento de verdad, he hecho pruebas en dos entornos:

1. Mi ordenador personal.
2. El clúster atcgrid.

Para no tener que estar lanzando ejecuciones a mano en el clúster, he dejado preparado un script de SLURM (mandelbrot.sh). Con este script es comodísimo cambiar cuántos nodos o procesos por nodo quiero usar, y me vuelca todos los tiempos de ejecución en un archivo de texto limpio para poder analizar los datos luego.

- **Lanzar a la cola de SLURM:** sbatch mandelbrot.sh

```
#!/bin/bash
#SBATCH --job-name=mandelbrot_mpi
#SBATCH --partition=acap
#SBATCH --nodes=1             # <-- CAMBIAR: número de nodos (1, 2, 3)
#SBATCH --ntasks-per-node=24  # <-- CAMBIAR: procesos por nodo (8, 16, 24)
#SBATCH --cpus-per-task=1
#SBATCH --time=00:05:00
#SBATCH --output=salida_mandelbrot_%j.txt
#SBATCH --error=error_mandelbrot_%j.txt

# Procesos totales = --nodes * --ntasks-per-node
echo "=== Generacion Fractal Mandelbrot 4K ==="
echo "Nodos: $SLURM_NNODES  |  Procesos totales: $SLURM_NTASKS"

mpirun -np $SLURM_NTASKS ./paralelo
```

4. Resultados y Análisis de Rendimiento

4.1. Rendimiento Secuencial: PC vs. Clúster

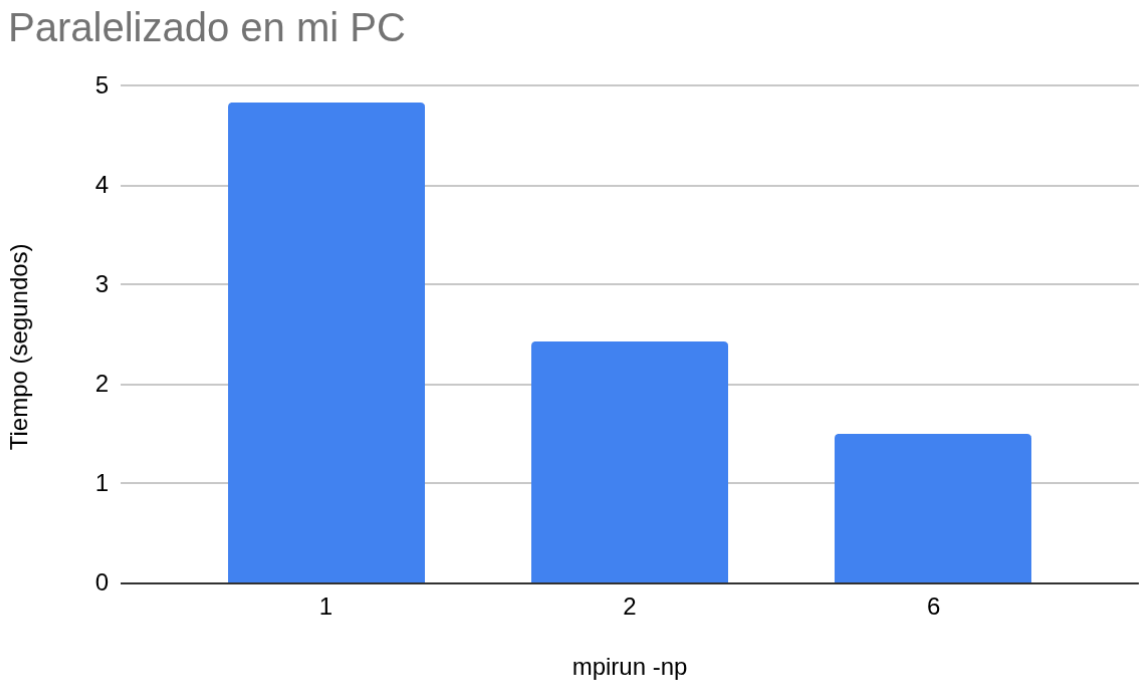
Lo primero fue medir cuánto tarda el código sin paralelizar sacando una imagen 4K con 1000 iteraciones de límite. En mi PC tardó **4,85 segundos**, mientras que el nodo del clúster tardó **6,38 segundos**. Es un comportamiento súper habitual: los procesadores de nuestros ordenadores domésticos suelen tener una frecuencia de reloj más alta por núcleo, por lo que en tareas puramente monohilo suelen ser más rápidos que los de un servidor.

mandelbrot.c	tiempo pc	tiempo atcgrid	
Secuencial	4,850450	6,387610	segundos

4.2. Escalabilidad Paralela en PC

Antes de pasar al clúster, comprobé que la paralelización de la memoria funcionaba correctamente en mi equipo y los tiempos variando los nucleos:

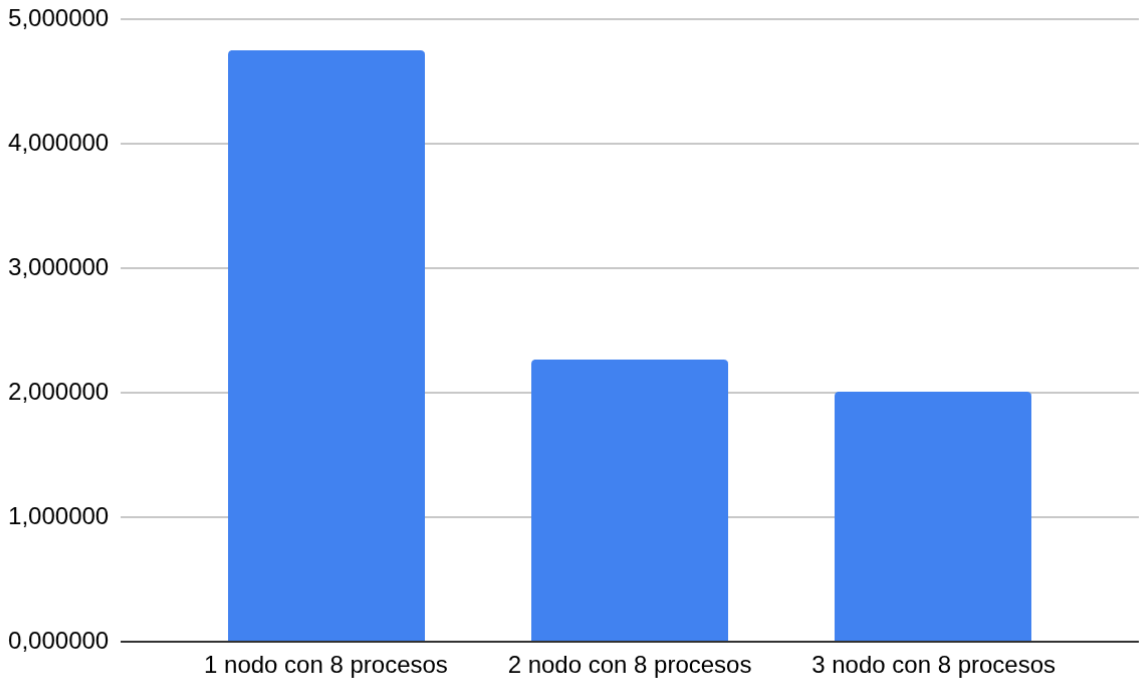
Paralelizado	mpicxx -Wall -O3 mandelbrot_paralelo.cpp -o paralelo
mpirun -np	
1	4,84046
2	2,430380
6	1,50638

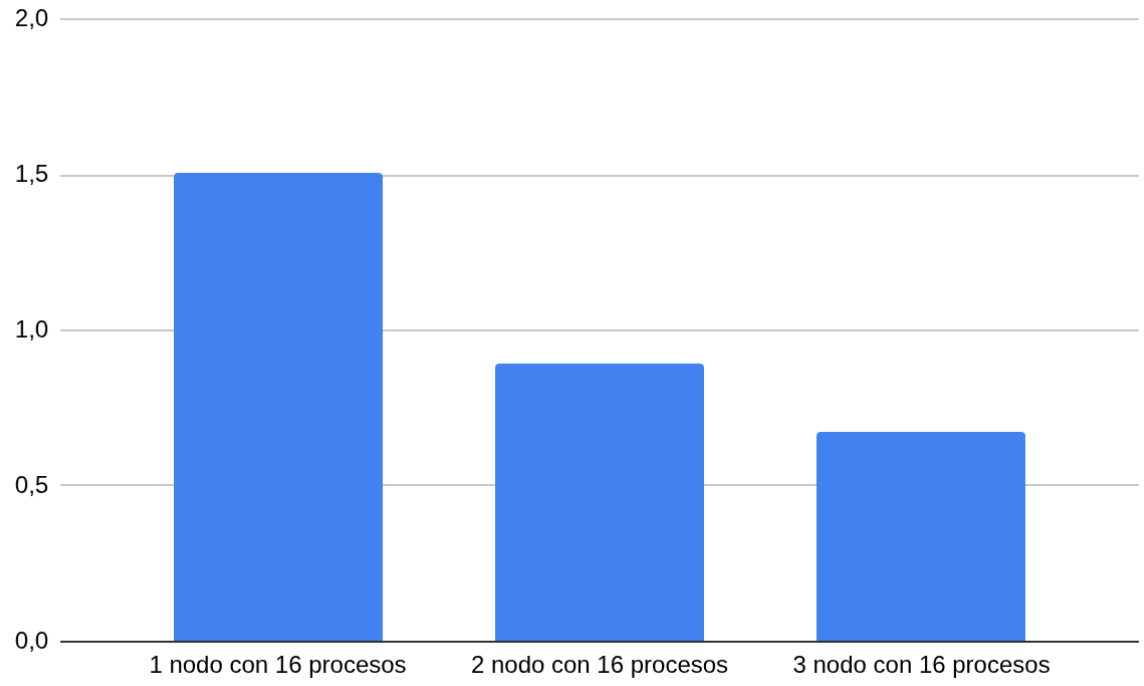


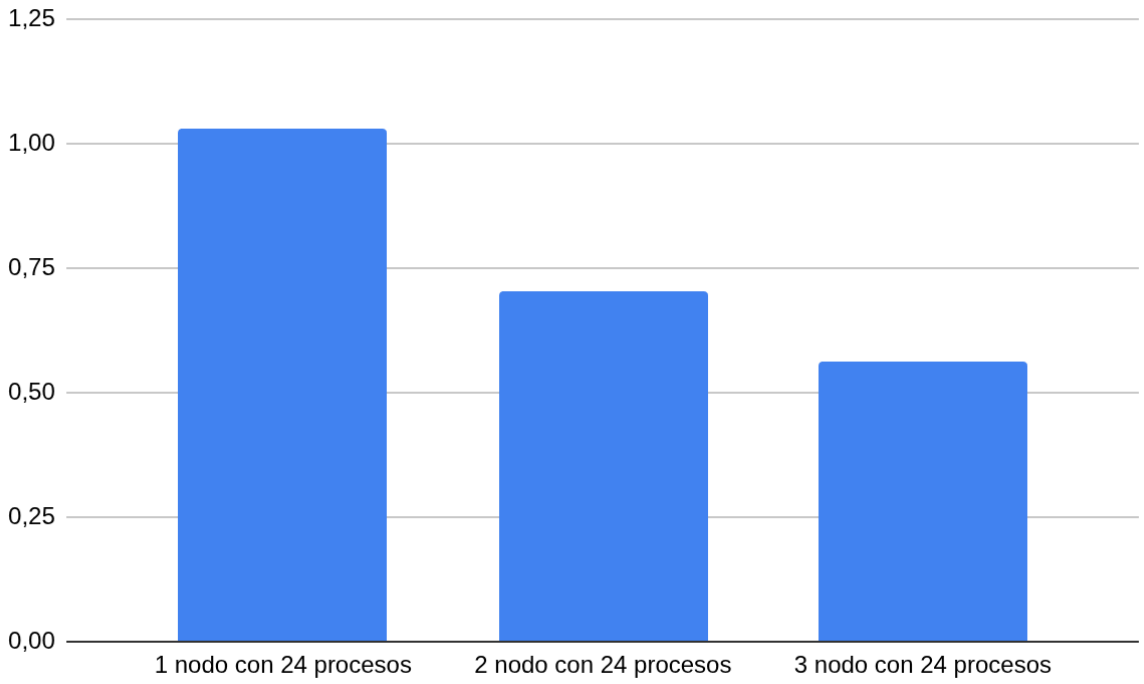
4.3. Escalabilidad Paralela en atcgrid

Una vez llevado el código MPI al clúster, la mejora de rendimiento al usar varios nodos es espectacular. Fijando los procesos por nodo (en 8, en 16 y en 24) y aumentando el número de nodos, los tiempos caen en picado.

1 nodo con 24 procesos	1,03078	1 nodo con 16 procesos	1,50805	1 nodo con 8 procesos	4,758160
2 nodo con 24 procesos	0,701685	2 nodo con 16 procesos	0,894956	2 nodo con 8 procesos	2,27099
3 nodo con 24 procesos	0,56382	3 nodo con 16 procesos	0,671997	3 nodo con 8 procesos	2,01176


5,000000
4,000000
3,000000
2,000000
1,000000
0,000000
1 nodo con 8 procesos
2 nodo con 8 procesos
3 nodo con 8 procesos


2,0
1,5
1,0
0,5
0,0
1 nodo con 16 procesos
2 nodo con 16 procesos
3 nodo con 16 procesos

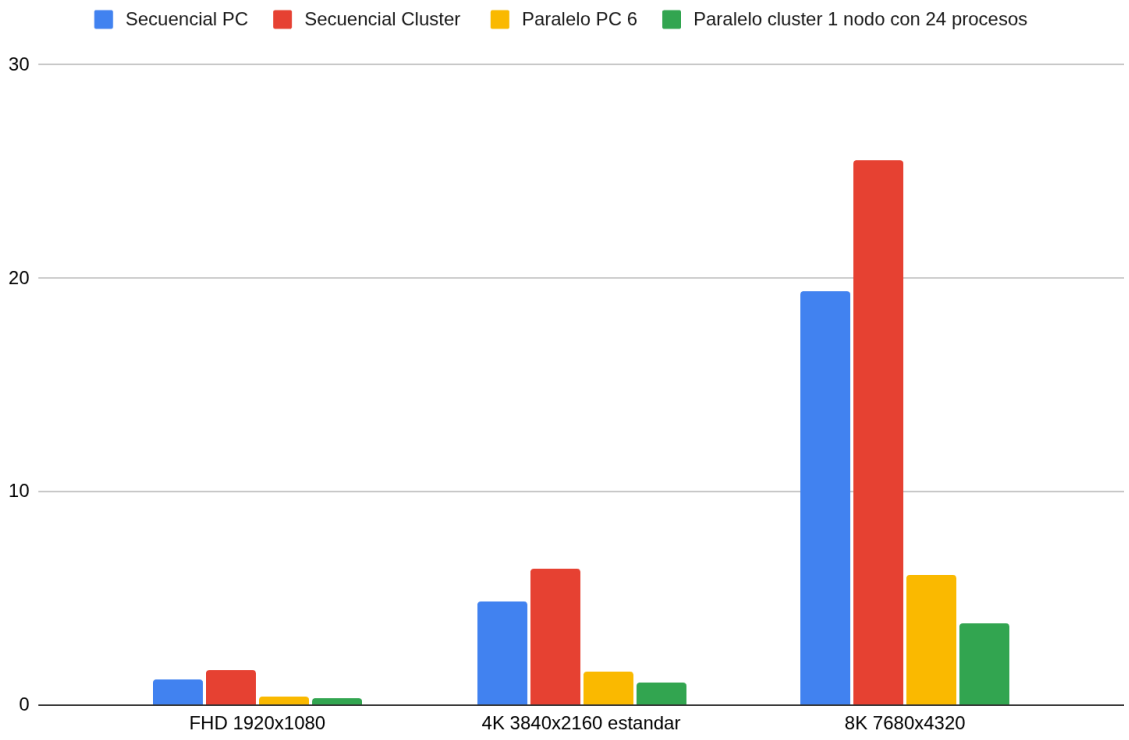


El tiempo baja de unos 4,7 segundos usando un solo nodo a solo 0,6 segundos usando 3 nodos. Esto demuestra una cosa muy importante: la latencia y el tiempo que tarda la red del clúster en mover la imagen pesada con MPI_Gather es insignificante en comparación con todo el tiempo de CPU que ganamos al repartir la carga matemática.

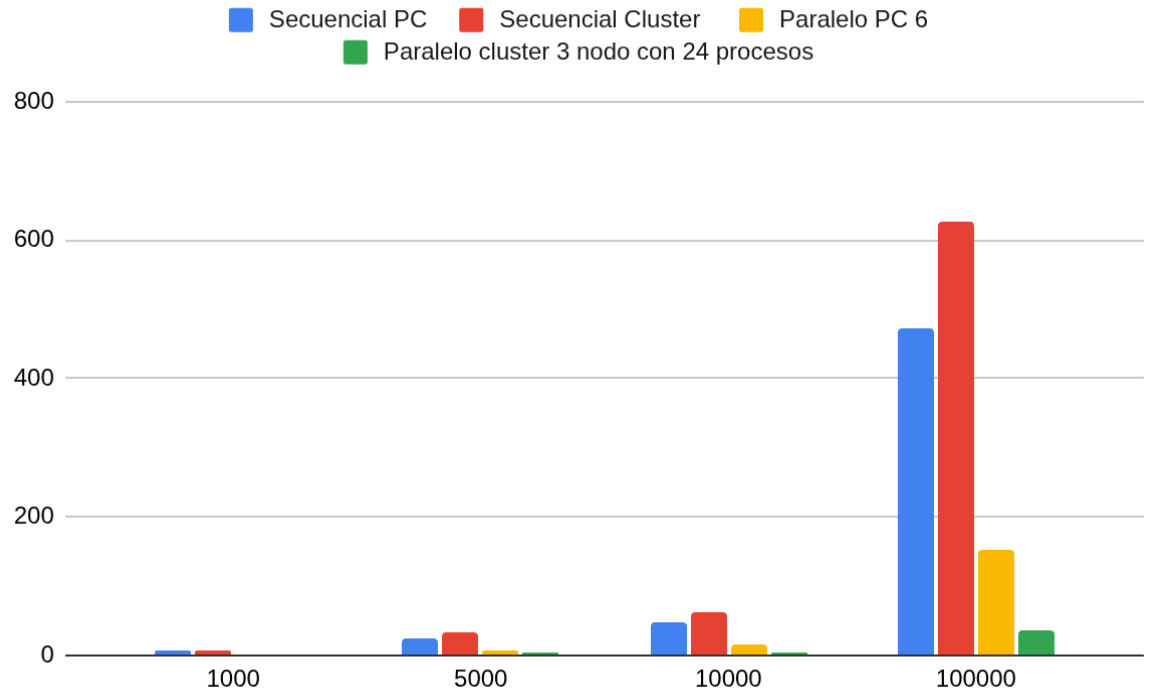
4.4. Análisis de Granularidad (Resolución e Iteraciones)

Para estresar un poco más el sistema, quise probar qué pasaba al variar muchísimo la carga de trabajo, ya sea cambiando la resolución de la imagen (desde FHD hasta 8K) o subiendo el límite matemático (max_iter de 1.000 hasta 100.000).

Lo interesante de esto es que, si aumentas la resolución, obligas a la red a transmitir muchos más megas de datos (sufre el paso de mensajes). En cambio, si subes el max_iter, la imagen ocupa exactamente la misma memoria en MB, pero saturas a los procesadores haciendo multiplicaciones.



	Secuencial PC	Secuencial Cluster	Paralelo PC 6	Paralelo cluster 1 nodo con 24 procesos
FHD 1920x1080	1,22071	1,59842	0,388432	0,278301
4K 3840x2160 estandar	4,85572	6,38761	1,51797	1,03562
8K 7680x4320	19,4015	25,5635	6,06494	3,78732



max_iter	Secuencial PC	Secuencial Cluster	Paralelo PC 6	Paralelo cluster 3 nodo con 24 procesos
1000	4,85612	6,41141	1,51078	0,54771
5000	23,7372	31,4342	7,47333	1,93938
10000	47,4411	62,7128	15,0385	3,67658
100000	470,915	625,263	151,021	34,7246

Los gráficos muestran que, incluso con resoluciones de 8K o forzando la convergencia a 100.000 iteraciones, el uso de varios nodos en el clúster pulveriza los tiempos de mi ordenador secuencial. El *overhead* (el coste adicional) de montar el entorno paralelo queda totalmente compensado.

5. Conclusiones

Generar fractales es, sin duda, un problema "vergonzosamente paralelo" (*embarrassingly parallel*) y un ejemplo de manual para entender cómo repartir datos. A diferencia de otras prácticas donde se acumulan resultados numéricos simples, aquí el principal reto técnico ha sido entender cómo particionar correctamente la memoria y utilizar MPI_Gather para reensamblarla después.

Los tiempos de ejecución dejan claro que, siempre y cuando la cantidad de cálculos por píxel sea alta, las redes de los clústeres modernos como atcgrid soportan sin problema el envío de estructuras de datos grandes, dándonos una aceleración (*speedup*) altísima.